

SQL - výběr a filtrování dat

Co je výběr dat v SQL

- výběr dat znamená získání dat z databáze pomocí dotazu
- v SQL se k tomu používá hlavně příkaz `SELECT`
- výsledek dotazu je tabulka
- výsledek může obsahovat:
 - všechny sloupce
 - jen vybrané sloupce
 - vypočítané hodnoty
 - agregované hodnoty
 - data z více tabulek

```
SELECT Jmeno, Prijmeni  
FROM Studenti;
```

Význam:

- z tabulky `Studenti` vybereme sloupce `Jmeno` a `Prijmeni`

Základní struktura SELECT dotazu

Základní tvar

```
SELECT sloupce  
FROM tabulka;
```

Příklad:

```
SELECT Jmeno, Prijmeni  
FROM Studenti;
```

SELECT

- určuje, jaké sloupce chceme zobrazit
- může obsahovat:
 - názvy sloupců
 - výpočty
 - funkce
 - aliasy

Příklad výběru konkrétních sloupců:

```
SELECT Jmeno, Prijmeni, Vek  
FROM Studenti;
```

SELECT *

- hvězdička znamená všechny sloupce
- používá se hlavně pro rychlé testování
- v reálných aplikacích je často lepší vypsát konkrétní sloupce

Příklad:

```
SELECT *  
FROM Studenti;
```

Nevýhody SELECT * :

- může vracet zbytečně moc dat
- je méně přehledný
- může být pomalejší
- aplikace může být závislá na pořadí nebo počtu sloupců

FROM

- určuje, z jaké tabulky data vybíráme

Příklad:

```
SELECT *  
FROM Studenti;
```

Alias

Co je alias

- alias je dočasné pojmenování sloupce nebo tabulky
- používá se pro lepší čitelnost
- zapisuje se pomocí AS
- u tabulek se často používá zkrácený název

Alias sloupce

```
SELECT Jmeno AS KrestniJmeno
```

```
FROM Studenti;
```

- Výsledek bude mít sloupec pojmenovaný `KrestniJmeno` .

Alias tabulky

```
SELECT s.Jmeno, s.Prijmeni  
FROM Studenti AS s;
```

Význam:

- `s` je zkratka pro tabulku `Studenti`

Zkrácený zápis:

```
SELECT s.Jmeno, s.Prijmeni  
FROM Studenti s;
```

WHERE

Co je `WHERE`

- `WHERE` slouží k filtrování řádků
- určuje podmínku, kterou musí řádek splnit
- používá se u:
 - `SELECT`
 - `UPDATE`
 - `DELETE`

Příklad:

```
SELECT *  
FROM Studenti  
WHERE Vek >= 18;
```

Význam:

- vybere jen studenty, kterým je alespoň 18 let

Operátory ve `WHERE`

Porovnávací operátory

Operátor	Význam
=	rovná se
<>	nerovná se
!=	nerovná se
>	větší než
<	menší než
>=	větší nebo rovno
<=	menší nebo rovno

Příklad:

```
SELECT *
FROM Studenti
WHERE Vek > 18;
```

Logické operátory

Operátor	Význam
AND	obě podmínky musí platit
OR	stačí jedna podmínka
NOT	negace podmínky

Příklad s AND :

```
SELECT *
FROM Studenti
WHERE Vek >= 18 AND Trida = '4.A';
```

Příklad s OR :

```
SELECT *
FROM Studenti
WHERE Trida = '4.A' OR Trida = '4.B';
```

Příklad s NOT :

```
SELECT *
FROM Studenti
WHERE NOT Trida = '4.A';
```

Další operátory ve WHERE

BETWEEN

- vybírá hodnoty v rozsahu
- hranice jsou obvykle zahrnuté

Příklad:

```
SELECT *  
FROM Produkty  
WHERE Cena BETWEEN 100 AND 500;
```

Význam:

- vybere produkty s cenou od 100 do 500 včetně

IN

- kontroluje, jestli je hodnota v seznamu hodnot

Příklad:

```
SELECT *  
FROM Studenti  
WHERE Trida IN ('4.A', '4.B', '4.C');
```

Význam:

- vybere studenty ze tříd 4.A, 4.B nebo 4.C

LIKE

- používá se pro hledání podle části textu
- často se používají zástupné znaky

Nejčastější zástupné znaky:

Znak	Význam
%	libovolný počet znaků
_	právě jeden znak

Příklad:

```
SELECT *  
FROM Studenti
```

```
WHERE Prijmeni LIKE 'Nov%';
```

Význam:

- najde příjmení začínající na Nov
- například Novák , Novotný , Nováková

Příklad:

```
SELECT *  
FROM Studenti  
WHERE Jmeno LIKE '_an';
```

Význam:

- najde třípísmenná jména končící na an
- například Jan

IS NULL

- používá se pro kontrolu hodnoty NULL
- NULL znamená chybějící nebo neznámá hodnota

Příklad:

```
SELECT *  
FROM Studenti  
WHERE Email IS NULL;
```

Význam:

- vybere studenty, kteří nemají vyplněný e-mail

IS NOT NULL

```
SELECT *  
FROM Studenti  
WHERE Email IS NOT NULL;
```

Význam:

- vybere studenty, kteří mají vyplněný e-mail

Pozor:

```
WHERE Email = NULL
```

je špatně.

Správně:

```
WHERE Email IS NULL
```

JOIN

Co je JOIN

- JOIN slouží ke spojení více tabulek
- používá se, když potřebujeme data z více tabulek najednou
- tabulky se obvykle spojují přes:
 - primární klíč
 - cizí klíč

Příklad situace:

- tabulka `Studenti` obsahuje studenty
- tabulka `Tridy` obsahuje názvy tříd
- student má v sobě `TridaId`
- chceme vypsat jméno studenta i název jeho třídy

Příklad tabulek pro JOIN

Tabulka `Studenti`:

Id	Jmeno	TridaId
1	Jan	1
2	Eva	2
3	Petr	NULL
4	Anna	5

Tabulka `Tridy`:

Id	Nazev
1	4.A
2	4.B
3	4.C

INNER JOIN

Co je INNER JOIN

- vrátí jen řádky, které mají odpovídající záznam v obou tabulkách
- pokud student nemá existující třídu, ve výsledku nebude

Příklad:

```
SELECT Studenti.Jmeno, Tridy.Nazev
FROM Studenti
INNER JOIN Tridy ON Studenti.TridaId = Tridy.Id;
```

Výsledek:

Jmeno	Nazev
Jan	4.A
Eva	4.B

Nevrátí se:

- Petr, protože má `TridaId = NULL`
- Anna, protože má `TridaId = 5`, ale taková třída neexistuje

LEFT JOIN

Co je LEFT JOIN

- vrátí všechny řádky z levé tabulky
- pokud se nenajde odpovídající řádek v pravé tabulce, doplní se `NULL`

Příklad:

```
SELECT Studenti.Jmeno, Tridy.Nazev
FROM Studenti
LEFT JOIN Tridy ON Studenti.TridaId = Tridy.Id;
```

Výsledek:

Jmeno	Nazev
Jan	4.A
Eva	4.B
Petr	NULL
Anna	NULL

Použití:

- když chceme všechny studenty, i ty bez přiřazené třídy

RIGHT JOIN

Co je RIGHT JOIN

- vrátí všechny řádky z pravé tabulky
- pokud se nenajde odpovídající řádek v levé tabulce, doplní se NULL
- v praxi se používá méně často
- často se dá nahradit pomocí LEFT JOIN s opačným pořadím tabulek

Příklad:

```
SELECT Studenti.Jmeno, Tridy.Nazev
FROM Studenti
RIGHT JOIN Tridy ON Studenti.TridaId = Tridy.Id;
```

Význam:

- vrátí všechny třídy, i ty, ve kterých není žádný student

FULL OUTER JOIN

Co je FULL OUTER JOIN

- vrátí všechny odpovídající i neodpovídající řádky z obou tabulek
- kde chybí dvojice, doplní se NULL
- ne všechny databáze ho podporují

Příklad:

```
SELECT Studenti.Jmeno, Tridy.Nazev
FROM Studenti
FULL OUTER JOIN Tridy ON Studenti.TridaId = Tridy.Id;
```

Význam:

- vrátí studenty s třídou
- studenty bez třídy
- třídy bez studentů

CROSS JOIN

Co je CROSS JOIN

- vytvoří kartézský součin
- každý řádek z první tabulky spojí s každým řádkem z druhé tabulky
- používá se jen ve speciálních případech

Příklad:

```
SELECT Studenti.Jmeno, Tridy.Nazev  
FROM Studenti  
CROSS JOIN Tridy;
```

Pokud má tabulka:

- 4 studenty
- 3 třídy

Výsledek bude mít:

$4 \times 3 = 12$ řádků

Pozor:

- nechtěný kartézský součin může vytvořit obrovský počet řádků
- proto se běžně používá `JOIN ... ON`

FROM s více tabulkami

Starší zápis:

```
SELECT *  
FROM Studenti, Tridy;
```

Problém:

- bez podmínky vznikne kartézský součin
- spojí se každý student s každou třídou
- to většinou nechceme

Lepší zápis:

```
SELECT Studenti.Jmeno, Tridy.Nazev  
FROM Studenti  
INNER JOIN Tridy ON Studenti.TridaId = Tridy.Id;
```

ORDER BY

Co je ORDER BY

- slouží k seřazení výsledků
- používá se na konci dotazu
- řadit lze podle jednoho nebo více sloupců

ASC

- vzestupné řazení
- od nejmenšího po největší
- od A do Z
- je výchozí, takže se nemusí psát

Příklad:

```
SELECT *  
FROM Studenti  
ORDER BY Prijmeni ASC;
```

DESC

- sestupné řazení
- od největšího po nejmenší
- od Z do A

Příklad:

```
SELECT *  
FROM Studenti  
ORDER BY Vek DESC;
```

Řazení podle více sloupců

```
SELECT *  
FROM Studenti  
ORDER BY Trida ASC, Prijmeni ASC;
```

Význam:

- nejdříve se řadí podle třídy
- v rámci stejné třídy podle příjmení

LIMIT, TOP a OFFSET

LIMIT

- omezuje počet vrácených řádků
- používá se například v MySQL, PostgreSQL nebo SQLite

Příklad:

```
SELECT *  
FROM Studenti  
LIMIT 10;
```

Význam:

- vrátí prvních 10 řádků

TOP

- obdoba LIMIT v Microsoft SQL Serveru
- píše se hned za SELECT

Příklad:

```
SELECT TOP 10 *  
FROM Studenti;
```

LIMIT s ORDER BY

Příklad:

```
SELECT *  
FROM Studenti  
ORDER BY Body DESC  
LIMIT 3;
```

Význam:

- seřadí studenty podle bodů sestupně
- vrátí 3 nejlepší studenty

OFFSET

- přeskočí určitý počet řádků
- používá se například pro stránkování

Příklad:

```
SELECT *  
FROM Studenti
```

```
ORDER BY Id
LIMIT 10 OFFSET 20;
```

Význam:

- přeskočí prvních 20 řádků
- vrátí dalších 10 řádků

Použití:

- stránkování výsledků
- například třetí stránka po 10 záznamech

GROUP BY

Co je GROUP BY

- seskupuje řádky podle stejné hodnoty
- používá se s agregačními funkcemi
- z více řádků vytvoří skupiny

Příklad:

```
SELECT Trida, COUNT(*) AS PocetStudentu
FROM Studenti
GROUP BY Trida;
```

Význam:

- studenti se rozdělí podle třídy
- pro každou třídu se spočítá počet studentů

Agregační funkce

Co jsou agregační funkce

- funkce, které počítají výsledek z více řádků
- často se používají s GROUP BY

Funkce	Význam
COUNT()	počet
SUM()	součet
AVG()	průměr
MIN()	minimum

Funkce	Význam
MAX()	maximum

COUNT

```
SELECT COUNT(*) AS PocetStudentu
FROM Studenti;
```

Význam:

- spočítá počet všech studentů

SUM

```
SELECT SUM(Cena) AS CelkovaCena
FROM ObjednavkaPolozka;
```

Význam:

- sečte ceny všech položek

AVG

```
SELECT AVG(Vek) AS PrumernyVek
FROM Studenti;
```

Význam:

- spočítá průměrný věk studentů

MIN a MAX

```
SELECT MIN(Vek) AS Nejmladsi, MAX(Vek) AS Nejstarsi
FROM Studenti;
```

Význam:

- najde nejnižší a nejvyšší věk

HAVING

Co je HAVING

- filtruje skupiny po použití GROUP BY
- je podobný WHERE

- rozdíl je v tom, že `WHERE` filtruje jednotlivé řádky
- `HAVING` filtruje celé skupiny

Příklad:

```
SELECT Trida, COUNT(*) AS PocetStudentu
FROM Studenti
GROUP BY Trida
HAVING COUNT(*) > 20;
```

Význam:

- vypíše jen třídy, které mají více než 20 studentů

WHERE vs. HAVING

WHERE	HAVING
filtruje řádky	filtruje skupiny
používá se před <code>GROUP BY</code>	používá se po <code>GROUP BY</code>
neumí filtrovat podle výsledku agregace	používá se s agregačními funkcemi
například <code>Vek > 18</code>	například <code>COUNT(*) > 20</code>

Příklad obojího najednou:

```
SELECT Trida, COUNT(*) AS PocetStudentu
FROM Studenti
WHERE Aktivni = 1
GROUP BY Trida
HAVING COUNT(*) > 20;
```

Význam:

- nejdřív se vyberou jen aktivní studenti
- potom se seskupí podle třídy
- nakonec se nechají jen třídy s více než 20 aktivními studenty

DISTINCT

Co je `DISTINCT`

- odstraní duplicitní hodnoty z výsledku
- vrátí jen unikátní hodnoty

Příklad:

```
SELECT DISTINCT Trida
FROM Studenti;
```

Význam:

- vypíše každou třídu jen jednou

Poddotazy

Co je poddotaz

- poddotaz = dotaz uvnitř jiného dotazu
- používá se, když potřebujeme výsledek jednoho dotazu použít v jiném dotazu

Příklad:

```
SELECT *
FROM Studenti
WHERE Vek > (
    SELECT AVG(Vek)
    FROM Studenti
);
```

Význam:

- vybere studenty, kteří jsou starší než průměrný věk studentů

Pořadí zápisu SELECT dotazu

Typické pořadí zápisu:

```
SELECT sloupce
FROM tabulka
JOIN dalsi_tabulka ON podminka
WHERE podminka_radku
GROUP BY sloupce
HAVING podminka_skupin
ORDER BY sloupce
LIMIT pocet;
```

Příklad:

```
SELECT Tridy.Nazev, COUNT(Studenti.Id) AS PocetStudentu
FROM Tridy
LEFT JOIN Studenti ON Studenti.TridaId = Tridy.Id
WHERE Studenti.Aktivni = 1
```



```
GROUP BY Tridy.Nazev
HAVING COUNT(Studenti.Id) > 20
ORDER BY PocetStudentu DESC
LIMIT 5;
```

Pořadí zpracování dotazu databází

- SQL dotaz se píše v jiném pořadí, než se logicky zpracovává.

Logické pořadí zpracování

1. FROM
2. JOIN
3. WHERE
4. GROUP BY
5. HAVING
6. SELECT
7. ORDER BY
8. LIMIT

Význam:

- nejdříve se najdou tabulky
- potom se spojí
- pak se vyfiltrují řádky
- potom se vytvoří skupiny
- pak se vyfiltrují skupiny
- potom se vyberou sloupce
- nakonec se výsledek seřadí a omezí

Praktické příklady

1. Vypsání všech studentů

```
SELECT *
FROM Studenti;
```

2. Vypsání jmen a příjmení

```
SELECT Jmeno, Prijmeni
FROM Studenti;
```

3. Studenti starší než 18 let

```
SELECT *  
FROM Studenti  
WHERE Vek > 18;
```

4. Studenti ze tříd 4.A a 4.B

```
SELECT *  
FROM Studenti  
WHERE Trida IN ('4.A', '4.B');
```

5. Studenti, kterým příjmení začíná na Nov

```
SELECT *  
FROM Studenti  
WHERE Prijmeni LIKE 'Nov%';
```

6. Studenti bez e-mailu

```
SELECT *  
FROM Studenti  
WHERE Email IS NULL;
```

7. Studenti seřazení podle příjmení

```
SELECT *  
FROM Studenti  
ORDER BY Prijmeni ASC;
```

8. Tři nejlepší studenti podle bodů

```
SELECT *  
FROM Studenti  
ORDER BY Body DESC  
LIMIT 3;
```

9. Student a název jeho třídy

```
SELECT Studenti.Jmeno, Studenti.Prijmeni, Tridy.Nazev  
FROM Studenti  
INNER JOIN Tridy ON Studenti.TridaId = Tridy.Id;
```

10. Všichni studenti i bez třídy

```
SELECT Studenti.Jmeno, Studenti.Prijmeni, Tridy.Nazev
FROM Studenti
LEFT JOIN Tridy ON Studenti.TridaId = Tridy.Id;
```

11. Počet studentů ve třídách

```
SELECT Trida, COUNT(*) AS PocetStudentu
FROM Studenti
GROUP BY Trida;
```

12. Třídy s více než 20 studenty

```
SELECT Trida, COUNT(*) AS PocetStudentu
FROM Studenti
GROUP BY Trida
HAVING COUNT(*) > 20;
```

13. Průměrný věk podle třídy

```
SELECT Trida, AVG(Vek) AS PrumernyVek
FROM Studenti
GROUP BY Trida;
```

14. Aktivní studenti podle třídy, jen větší třídy

```
SELECT Trida, COUNT(*) AS PocetStudentu
FROM Studenti
WHERE Aktivni = 1
GROUP BY Trida
HAVING COUNT(*) > 20
ORDER BY PocetStudentu DESC;
```